10 Python Programming Problem Statements

*Covering all core Python concepts including Variables · Datatypes · Literals · Base Conversions · Type Conversions · Arithmetic · Bitwise · Logical · Comparison Operators · if/if-else/if-elif-else · match-case · Lists · Tuples · Sets · Dictionaries · for · while · break · continue · pass · List Comprehension · String Methods · Functions (Positional-only, Keyword-only, Keyword, Positional, Mixed, *args) · Exception Handling (try-except)*

Table of Contents

01	Smart Expense Tracker
02	Cryptographic Text Processor
03	Student Performance Analytics Engine
04	Inventory & Supply Chain Simulator
05	Personal Finance Planner
06	Text-Based RPG Game Engine
07	Data Pipeline & Transformation Tool
08	Smart Library Management System
09	Scientific Calculator & Unit Converter
10	Population & Census Data Analyser

Concept Coverage Matrix

Concept	Covered In
variables & datatypes	All 10
literals (int, float, bool, hex, bin)	All 10
base conversions (bin, oct, hex, int(,base))	#2, #7, #8, #9, #10
type conversions	All 10
arithmetic operators	All 10
bitwise operators (&, |, ^, ~, <<, >>)	#2, #4, #6, #7, #8, #9, #10
logical operators (and, or, not)	All 10
comparison operators	All 10
if condition	All 10
if-else condition	All 10
if-elif-else condition	All 10
match-case	All 10
lists & list methods	All 10
list comprehension	All 10
tuples	All 10
sets & set methods	All 10
dictionaries & dict methods	All 10
for loop	All 10
while loop	All 10
break	All 10
continue	All 10
pass	All 10
string methods (all major)	All 10
functions — positional-only args (/)	All 10
functions — keyword-only args (*)	All 10
functions — keyword arguments	All 10
functions — positional arguments	All 10
functions — mixed (positional-only + keyword-only)	All 10
functions — *args (var-positional)	#1, #2, #5, #7, #9, #10
exception handling — try-except	All 10

Problem
#01

Smart Expense Tracker

Concepts: variables · datatypes · literals · arithmetic · type conversion · comparison · logical · if-elif-else · lists · tuples · dictionaries · for · while · break · continue · pass · list comprehension · string methods · functions · keyword arguments · positional arguments · exception handling · try-except

Build a modular expense tracking system that records daily transactions, categorises spending, detects anomalies, and produces monthly summaries — weaving in every function argument style and exception-safe input handling throughout.

Key Tasks to Implement:

1.	Store expenses as a list of tuples: (date_str, category, amount_float, note_str). Use typed literals for defaults (0.0, '', True).
2.	add_expense(date, category, amount, /, *, note='', notify=True) — positional-only for core fields, keyword-only for optional metadata. Raise ValueError if amount <= 0; catch with try-except.
3.	categorise(expenses, *, min_amount=0, categories=None) — keyword-only filter; returns a list comprehension of matching tuples.
4.	Use a dictionary {category: [amounts]} to group expenses; compute totals with arithmetic operators.
5.	Apply type conversion: int() for day extraction from date string, float() for totals, str() for report lines.
6.	Use comparison operators (>, <, ==, !=) and logical operators (and, or, not) to flag high-value or duplicate entries.
7.	monthly_report(*months, currency='INR', show_zero=False) — var-positional + keyword; use if-elif-else to choose INR/USD/EUR symbol.
8.	String methods: strip(), split('-'), zfill(), center(), format(), ljust() to align the report table.
9.	while loop with break on 'quit' sentinel, continue on invalid category input, pass as stub for SMS-alert feature.
10.	Use match-case on category string to assign budget limits: 'food' → 5000, 'travel' → 10000, etc.
11.	List comprehension with nested if to extract over-budget items: [e for e in expenses if e[2] > limits[e[1]]].
12.	Functions: add_expense(), categorise(), compute_totals(), detect_anomalies(), monthly_report() — full modular design.

Problem
#02

Cryptographic Text Processor

Concepts: variables · datatypes · literals · base conversion · bitwise · type conversion · arithmetic · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · functions · keyword arguments · positional arguments · only positional args · only keyword args · exception handling · try-except

Implement a multi-cipher toolkit (Caesar, XOR, Vigenère, ROT13) with base-conversion output, full string-method input sanitisation, and exception-safe key validation. Every function argument style appears at least once.

Key Tasks to Implement:

1.	caesar_cipher(text, key, /, direction='encrypt') — positional-only args; shift with (ord(c)-65+key)%26 arithmetic; use isalpha(), isupper(), islower() to preserve case.
2.	xor_cipher(text, key, /) — positional-only; apply bitwise XOR (^) on ord() values; use bin(), hex() for multi-base output.
3.	vigenere(text, keyword, *, repeat=True, strict=False) — keyword-only options; use index %len(keyword) cycling with for loop.
4.	Store cipher lookup tables as dicts; use sets to find shared characters between plaintext and key alphabets.
5.	validate_key(key, /, *, min_len=1, allow_digits=False) — mixed args; raise TypeError and ValueError caught by caller's try-except.
6.	Use base conversion: display ciphertext ordinals in binary (0b), octal (0o), hex (0x) using type conversion.
7.	apply_all_ciphers(*texts, key, reverse=False) — var-positional + keyword-only; use list comprehension to apply each cipher.
8.	Bitwise checks: use & 0xFF to mask byte values, >> and << to simulate block cipher steps.
9.	String methods: translate(), maketrans(), encode('utf-8'), upper(), lower(), replace() for pre-processing.
10.	match-case on cipher_name string to dispatch to the right function; default branch raises NotImplementedError caught by try-except.
11.	while loop with break for interactive mode; continue to skip non-alpha characters; pass as placeholder for AES stub.
12.	Use logical operators: assert len(key) > 0 and key.isalpha() — catch AssertionError in outer try-except block.

Problem
#03

Student Performance Analytics Engine

Concepts: variables · datatypes · literals · arithmetic · type conversion · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · list comprehension · functions · positional arguments · keyword arguments · mixed arguments · exception handling · try-except

Analyse student exam data across subjects, compute ranks and letter grades, detect outliers, and generate formatted scorecards — using all argument styles and robust try-except blocks throughout.

Key Tasks to Implement:

1.	Data model: list of dicts {id, name, scores: {subject: mark}}. Marks as int literals; use float() conversion for GPA.
2.	add_student(student_id, name, /, scores=None, *, batch='A', active=True) — positional-only id+name, keyword-only metadata.
3.	compute_gpa(scores, /, *, scale=10.0, pass_mark=40) — positional-only scores dict; keyword-only config; arithmetic for average; if-elif-else for letter grade.
4.	rank_students(students, *, key='gpa', reverse=True) — keyword-only sorting params; use list.sort() with lambda; return sorted list.
5.	Use sets to find students who passed ALL subjects (set intersection with passing IDs per subject).
6.	detect_outliers(data, /, threshold=2.0) — positional-only; compute mean and std deviation using arithmetic; flag with comparison operators.
7.	Wrap all input parsing in try-except (ValueError, KeyError, TypeError) with meaningful error messages using f-strings.
8.	List comprehension: [s['name'] for s in students if compute_gpa(s['scores']) >= 7.0] — filter top performers.
9.	String methods: title() for name formatting, zfill(6) for ID padding, join() for subject list, center()/ljust() for scorecard alignment.
10.	match-case on grade letter: 'A' → 'Distinction', 'B' → 'First Class', etc.; default → 'Fail'.
11.	while loop driving interactive menu; break on '0' input; continue to re-prompt on invalid; pass as stub for email-dispatch feature.
12.	generate_scorecard(*student_ids, format='text', show_rank=True) — var-positional + keyword; build formatted report using string methods.

Problem
#04

Inventory & Supply Chain Simulator

Concepts: variables · datatypes · literals · arithmetic · bitwise · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · list comprehension · functions · positional arguments · keyword arguments · only positional args · only keyword args · exception handling · try-except

Simulate a warehouse inventory system supporting stock-in, stock-out, reorder alerts, and supplier management. Uses bitwise flags for product status, all function argument patterns, and full exception handling.

Key Tasks to Implement:

1.	Product: dict {sku, name, qty, price, status_flags(int), tags(set), history(list of tuples)}.
2.	add_product(sku, name, price, /, qty=0, *, category='general', reorder_level=10) — positional-only identity, keyword-only policy.
3.	Bitwise flags for status: bit 0 = active, bit 1 = perishable, bit 2 = imported. Use & to check, | to set, ~ to clear flags.
4.	stock_update(sku, delta, /, *, reason='manual', notify=False) — positional-only sku+delta; record history tuple (timestamp_str, delta, reason).
5.	Use try-except (KeyError) when accessing products dict; raise custom message if SKU not found.
6.	Reorder logic: if-elif-else on qty vs reorder_level; use comparison and logical operators (qty <= level and active_flag).
7.	search_products(*keywords, category=None, in_stock_only=True) — var-positional + keyword-only; list comprehension to filter matches.
8.	String methods: upper() for SKU normalisation, strip(), split(',') for bulk import, ljust()/rjust() for tabular display.
9.	match-case on update reason: 'sale', 'return', 'damage', 'restock' — update different counters in a stats dict.
10.	Use sets for tag management: product.tags | new_tags, product.tags & required_tags for compatibility checks.
11.	while loop for interactive CLI; break on exit command; continue on invalid SKU; pass as stub for barcode-scanner integration.
12.	generate_report(*, sort_by='qty', top_n=10, include_inactive=False) — all keyword-only; use sorted() with lambda and list slicing.

Problem
#05

Personal Finance Planner

Concepts: variables · datatypes · literals · arithmetic · type conversion · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · list comprehension · functions · keyword arguments · positional arguments · mixed arguments · exception handling · try-except

Build a personal budgeting tool that tracks income, expenses, savings goals, and EMI calculations. All arithmetic uses explicit operator precedence; every function demonstrates a distinct argument-passing style.

Key Tasks to Implement:

1.	Data structures: income_sources (dict), expense_log (list of tuples), goals (list of dicts), emi_plans (list of tuples).
2.	add_income(source, amount, /, *, recurring=True, tax_exempt=False) — positional-only core, keyword-only flags; validate with try-except ValueError.
3.	emi_calculator(principal, rate, tenure, /) — positional-only; formula: P*r*(1+r)^n / ((1+r)^n - 1) using arithmetic and ** operator.
4.	Use type conversion: int() for tenure in months, float() for rates, str() for formatted currency output.
5.	budget_check(*, month, year, strict=False) — all keyword-only; compute net = income - expenses using arithmetic; if-elif-else for surplus/deficit/balanced.
6.	Use comparison and logical operators to flag months where expenses > 80% of income AND savings < goal target.
7.	set_goal(name, target, /, deadline=None, *, priority=1, auto_invest=False) — mixed; store as dict in goals list.
8.	List comprehension: [g for g in goals if g['saved'] / g['target'] >= 0.5] — filter half-achieved goals.
9.	String methods: format() with f-strings for currency, strip() on user input, title() for goal names, center() for reports.
10.	match-case on goal['priority']: 1 → 'Critical', 2 → 'High', 3 → 'Medium', default → 'Low'.
11.	while loop for menu; break on exit; continue to re-prompt on non-numeric amount; pass as placeholder for UPI-sync feature.
12.	annual_summary(*years, include_projections=False, currency='INR') — var-positional + keyword; wrap entire body in try-except.

Problem
#06

Text-Based RPG Game Engine

Concepts: variables · datatypes · literals · arithmetic · bitwise · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · list comprehension · functions · keyword arguments · positional arguments · only positional args · only keyword args · mixed arguments · exception handling · try-except

A complete RPG engine with character creation, inventory, combat, and quest systems. Every concept appears naturally: bitwise flags for abilities, match-case for game commands, exception handling for invalid actions.

Key Tasks to Implement:

1.	Character: dict {name, hp, mp, level, stats(dict), inventory(list), abilities(int flags), quests(set)}.
2.	create_character(name, char_class, /, hp=100, mp=50, *, difficulty='normal', permadeath=False) — positional-only identity, keyword-only game config.
3.	Bitwise ability flags: bit 0=magic, bit 1=stealth, bit 2=strength. Use &, |, ^ to check/grant/toggle abilities.
4.	attack(attacker, defender, /, weapon=None, *, critical_chance=0.1, verbose=True) — mixed args; damage = base * multiplier using arithmetic; use comparison for hit/miss.
5.	Inventory: list of tuples (item_name, item_type, value, qty). Use list methods: append(), remove(), sort(), index().
6.	Use sets for quest tracking: completed_quests & available_quests for eligible quests; use logical operators for quest prerequisites.
7.	match-case on command string: 'attack', 'heal', 'inventory', 'quest', 'save', 'quit' — dispatches to respective functions.
8.	level_up(character, /, *, stat_boosts=None, announce=True) — mixed; use if-elif-else for milestone levels (10, 20, 50) granting special abilities.
9.	String methods: title() for item names, center() for battle display, ljust()/rjust() for stat tables, upper() for critical hit messages.
10.	List comprehension: [item for item in inventory if item[1]=='weapon' and item[2] > current_weapon_value].
11.	while game_loop: break on 'quit'; continue on invalid commands; pass as stub for save-to-file feature.
12.	Wrap all combat calculations in try-except (ZeroDivisionError, IndexError, KeyError); display user-friendly error messages.

Problem
#07

Data Pipeline & Transformation Tool

Concepts: variables · datatypes · literals · base conversion · type conversion · arithmetic · bitwise · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · list comprehension · functions · keyword arguments · positional arguments · only positional args · only keyword args · exception handling · try-except

Build a CSV-like data pipeline that reads raw string records, parses and validates them, applies transformations, performs aggregations, and outputs reports — all without any third-party library.

Key Tasks to Implement:

1.	Raw data: list of strings (CSV rows). Use string methods: split(','), strip(), replace(), lstrip(), rstrip() to parse each row.
2.	parse_record(raw_str, /, schema, *, strict=False, encoding='utf-8') — positional-only raw+schema; keyword-only flags; return dict or raise ValueError.
3.	Type conversion pipeline: int() for IDs, float() for prices, bool() for flags; wrap each in try-except with column name in message.
4.	Use a set to track seen IDs for deduplication; use set operations (difference, intersection) for join-like logic.
5.	transform(records, /, *transforms, inplace=False) — positional-only records, var-positional transform functions, keyword-only flag.
6.	Aggregation dict: {column: [values]}; compute sum, min, max, average using arithmetic and comparison operators.
7.	match-case on data type string ('int', 'float', 'str', 'bool', 'date') to dispatch the correct converter function.
8.	Use bitwise operations on integer fields: flag columns using &, |; extract bit ranges with >> and & mask for compact encoding.
9.	List comprehension with conditions: [r for r in records if r['price'] > 0 and r['qty'] >= 1 and r['active']] — multi-condition filter.
10.	Base conversion: display integer IDs in hex (hex()), binary (bin()), and octal (oct()) in debug output.
11.	pipeline(*stages, source=None, verbose=False) — var-positional stages + keyword-only config; chain stages with for loop; break on first error.
12.	String methods for output: join() for CSV re-serialisation, zfill(8) for padded IDs, format()/f-strings for reports.

Problem
#08

Smart Library Management System

Concepts: variables · datatypes · literals · arithmetic · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · list comprehension · functions · keyword arguments · positional arguments · only positional args · mixed arguments · exception handling · try-except

Manage a digital library's catalogue, member accounts, lending records, and overdue fines with full CRUD operations, all function argument patterns, and comprehensive exception handling.

Key Tasks to Implement:

1.	Book: dict {isbn, title, author, genre, copies, issued_to(set), tags(set)}. Member: dict {id, name, borrowed(list of tuples), fines(float)}.
2.	add_book(isbn, title, author, /, copies=1, *, genre='General', tags=None) — positional-only identity, keyword-only metadata; raise ValueError for duplicate ISBN in try-except.
3.	issue_book(member_id, isbn, /, *, due_days=14, override=False) — mixed; validate member and book with if-else; record (isbn, issue_date, due_date) tuple in member's list.
4.	Use sets: available_books = all_isbns - issued_isbns; member's borrowed set for O(1) lookup.
5.	compute_fine(member_id, isbn, /, return_date, *, rate_per_day=2.0) — positional-only IDs + date, keyword-only rate; arithmetic for days * rate; if-else for grace period.
6.	search(*, title=None, author=None, genre=None, available_only=False) — all keyword-only; list comprehension with logical operators to match multiple fields.
7.	String methods: lower() for case-insensitive search, strip() on all inputs, title() for display, join() for tag lists, ljust()/center() for catalogue table.
8.	match-case on command: 'issue', 'return', 'renew', 'search', 'fine', 'report' — route to respective functions.
9.	while True menu loop; break on 'exit'; continue on invalid member ID; pass as stub for barcode/QR scan integration.
10.	generate_report(*, report_type='overdue', sort_by='name', top_n=None) — all keyword-only; wrap in try-except (KeyError, TypeError).
11.	List comprehension: [(m['name'], len(m['borrowed'])) for m in members if len(m['borrowed']) > 3] — find heavy borrowers.
12.	Use logical operators: if not book['copies'] or member_id in book['issued_to'] — prevent double-issuing; if-elif-else for fine tier.

Problem
#09

Scientific Calculator & Unit Converter

Concepts: variables · datatypes · literals · base conversion · type conversion · arithmetic · bitwise · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · list comprehension · functions · keyword arguments · positional arguments · only positional args · only keyword args · mixed arguments · exception handling · try-except

A feature-rich calculator supporting arithmetic, bitwise, unit conversion, number-base conversion, and statistical functions — each feature implemented as a focused module demonstrating a distinct argument style.

Key Tasks to Implement:

1.	evaluate(expression_str, /, *, precision=6, degree_mode=False) — positional-only expression, keyword-only config; parse with split(); evaluate arithmetic using operator precedence rules.
2.	base_convert(value, from_base, to_base, /) — positional-only; use int(str, base) and conversion functions bin(), oct(), hex(); raise ValueError for unsupported base in try-except.
3.	bitwise_ops(a, b, /, op='and') — positional-only operands, keyword op; match-case on op: 'and'→&, 'or'→|, 'xor'→^, 'lshift'→<<, 'rshift'→>>.
4.	unit_convert(value, from_unit, to_unit, /, *, category='length') — mixed; dict of conversion factors; arithmetic for conversion; if-elif-else for category routing.
5.	statistics(*values, ddof=0, output='all') — var-positional for data, keyword-only config; compute mean, variance, std dev, median, mode without imports.
6.	Use type conversion throughout: float() for inputs, int() for base outputs, str() for formatted results, bool() for flag checks.
7.	Wrap all division operations in try-except ZeroDivisionError; wrap int(str, base) in try-except ValueError with informative message.
8.	history: list of tuples (operation_str, result, timestamp_str); use list methods: append(), pop(), reverse(), index().
9.	Use sets to track unique operations performed in session; use set methods: add(), discard(), difference().
10.	List comprehension: [round(v, precision) for v in values if isinstance(v, (int, float))] for result rounding.
11.	String methods: strip(), split(), replace() for expression parsing; format()/f-strings with precision specifiers for output.
12.	while REPL loop; match-case on command: 'calc', 'convert', 'bitwise', 'stats', 'history', 'clear', 'exit'; break/continue for control.

Problem
#10

Population & Census Data Analyser

Concepts: variables · datatypes · literals · arithmetic · bitwise · type conversion · comparison · logical · if-elif-else · match-case · for · while · break · continue · pass · lists · tuples · sets · dictionaries · string methods · list comprehension · functions · keyword arguments · positional arguments · only positional args · only keyword args · mixed arguments · exception handling · try-except · base conversion

The capstone problem — a full population data analysis platform covering every concept on the syllabus. Ingest raw census records, compute demographics, simulate growth, detect anomalies, and produce formatted multi-section reports.

Key Tasks to Implement:

1.	Census record: dict {region_id(hex str), name, population(int), area_km2(float), growth_rate(float), flags(int bitmask), history(list of tuples), categories(set)}.
2.	load_region(region_id, name, population, area, /, growth_rate=0.01, *, decade=2020, active=True) — positional-only identity+core, keyword-only temporal+status; use 0x literals for region_id; raise TypeError in try-except if types wrong.
3.	Bitmask flags: bit 0=urban, bit 1=coastal, bit 2=border, bit 3=capital. Use &, |, ^, ~ and >> to check/set/toggle/extract flags.
4.	simulate_growth(region, years, /, *, model='exponential', disease_year=None) — mixed; compute pop = P*(1+r)^t using ** arithmetic; if-elif-else for linear/exponential/logistic models.
5.	Use base conversion: display region_id in decimal, binary (bin()), octal (oct()) alongside hex in debug output.
6.	compute_density(population, area, /) — positional-only; wrap in try-except ZeroDivisionError; classify with if-elif-else: sparse/moderate/dense/mega.
7.	filter_regions(*flags_required, min_pop=0, max_density=None, categories=None) — var-positional flag bits + keyword-only thresholds; use list comprehension + logical operators.
8.	Type conversion pipeline: int() for populations, float() for rates, str() for IDs, bool() for flag bits, hex() for ID display.
9.	Use sets: urban_regions = {r['name'] for r in regions if r['flags'] & 1}; set operations for multi-criterion grouping.
10.	String methods: zfill(8) for region ID padding, center()/ljust() for report columns, format() for population with comma separators ('{:,}'.format()), upper() for region names.
11.	match-case on model type: 'exponential', 'linear', 'logistic', 'custom' — dispatch simulation; default raises NotImplementedError.
12.	List comprehension: [(r['name'], compute_density(r['population'], r['area'])) for r in regions if r['active'] and r['population'] > 1_000_000] — mega-city extraction.
13.	while simulation_running: break on convergence or year limit; continue to skip inactive regions; pass as placeholder for GIS map export.
14.	generate_report(regions, /, *, sections=('summary','growth','density'), output_format='text', top_n=5) — mixed; wrap in try-except; use all string methods for multi-section layout.
15.	Full modular design: load_region(), simulate_growth(), filter_regions(), compute_density(), compare_regions(), generate_report() — each in its own function with clear argument contracts.